ETH zürich

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

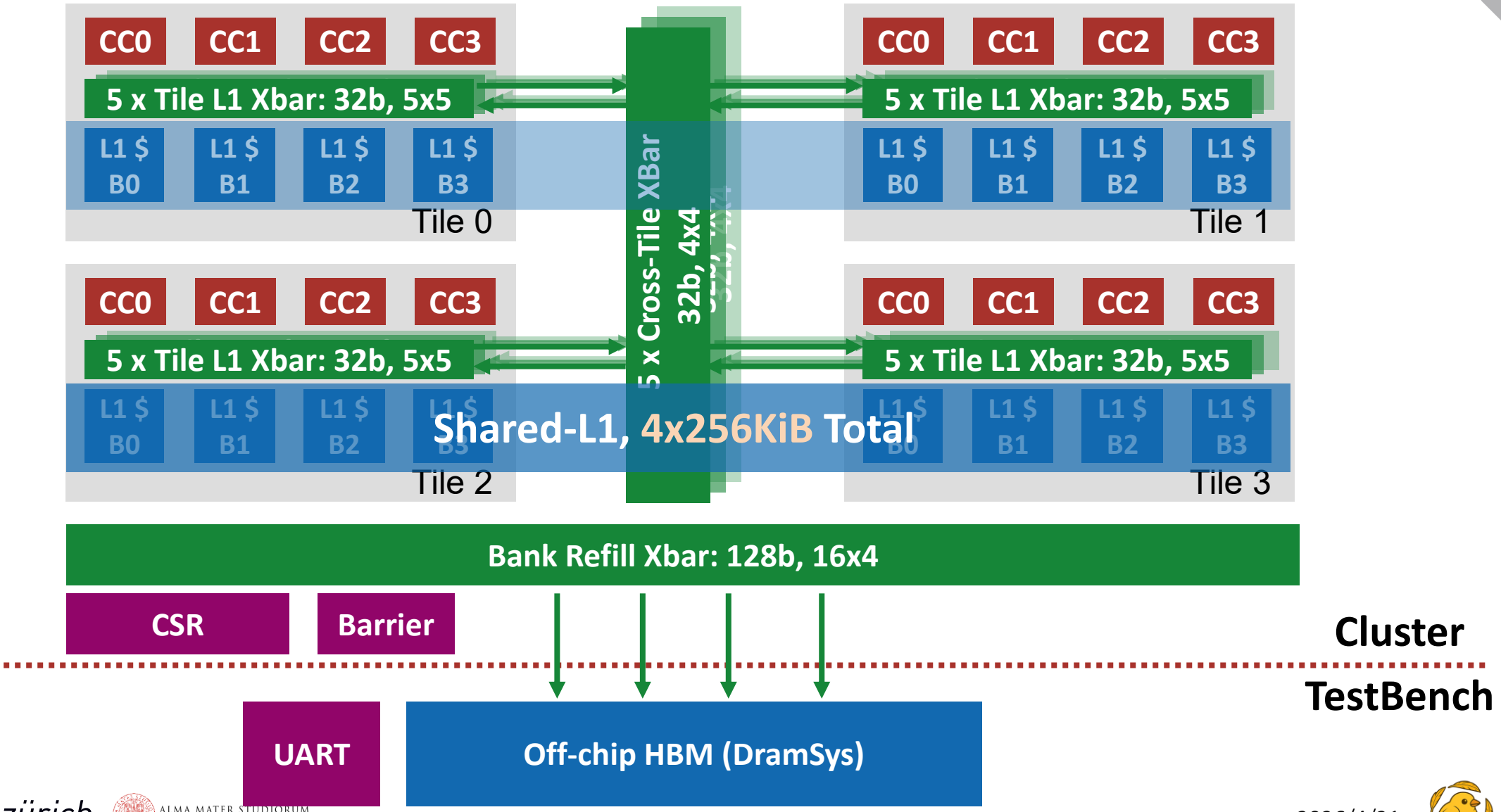
Zexin Fu, Diyou Shen zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli avanelli@iis.ee.ethz.ch
Luca Benini lbenini@iis.ee.ethz.ch

PULP Platform
Open Source Hardware, the way it should be!

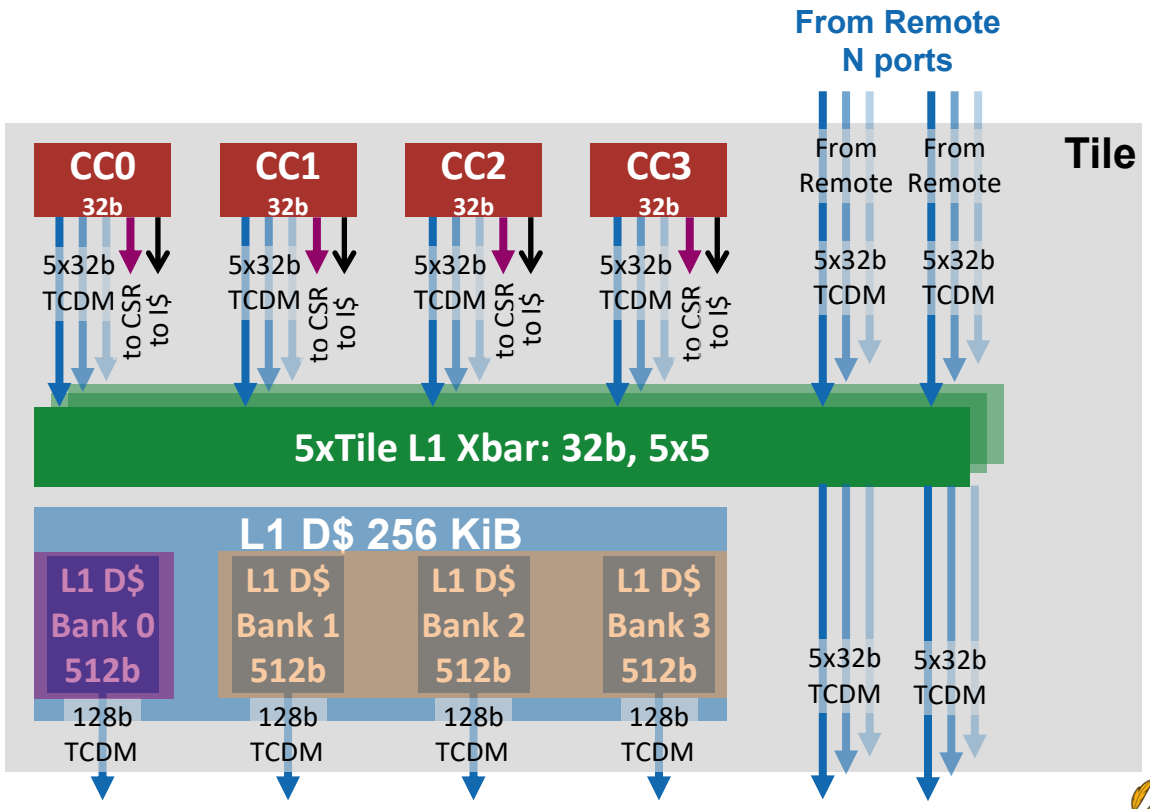
@pulp_platform
pulp-platform.org
youtube.com/pulp_platform

Hardware Development (Cluster)



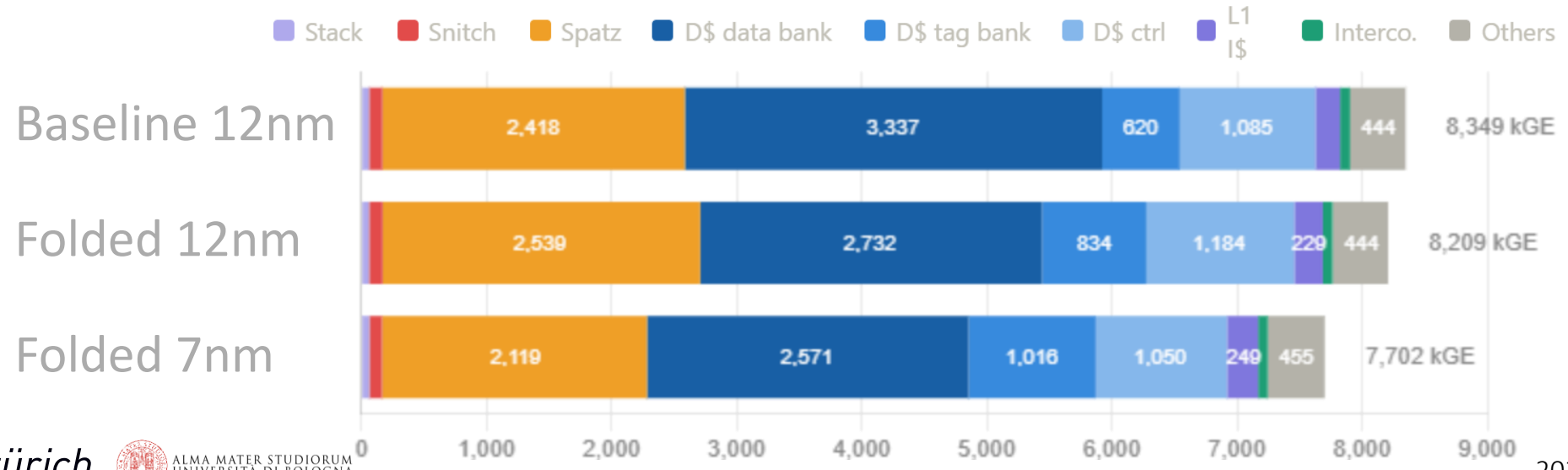
Hardware Development (Cluster)

- **Remote Bandwidth**
  - Now configurable from elaboration parameters (1, 2, 4)
  - Selected based on target tile
  - Preparations for multi-group
- **Return Code Support**
  - Add support for return code from SW
- **FFT Kernel**
  - Add FFT kernel for better test coverage
  - Not optimize for HW yet
  - Help to find and fix a bug in Spatz



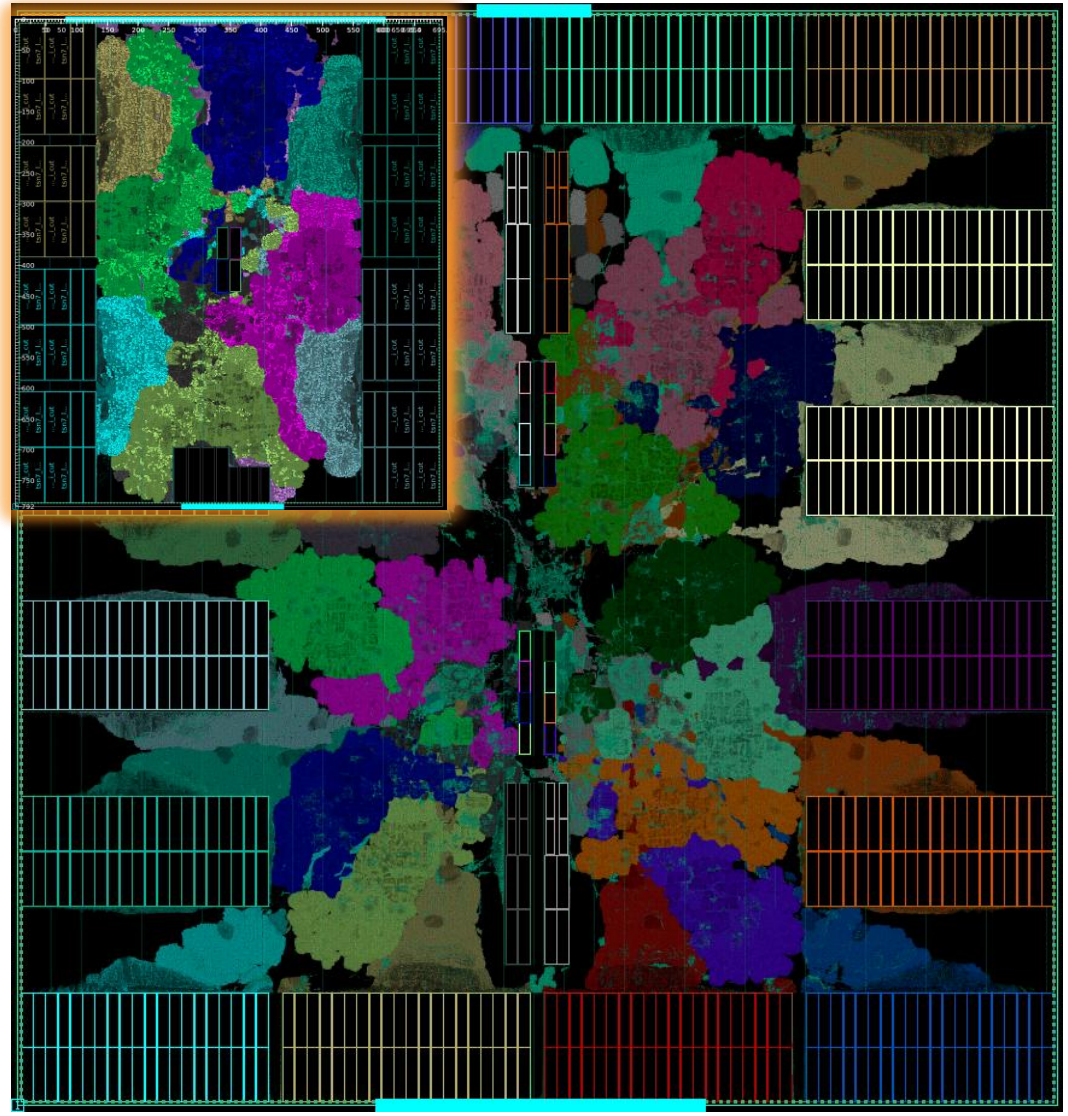
Hardware Development (Cluster)

- **Backend**
  - 12nm + 7nm
  - 1 GHz WW
- **Area Comparison (Folded vs Origin)**
  - Data SRAMs: -18% reduction (600 kGE) in GF12
  - Tag SRAMs: 34% increase (200 kGE) from byte-write support



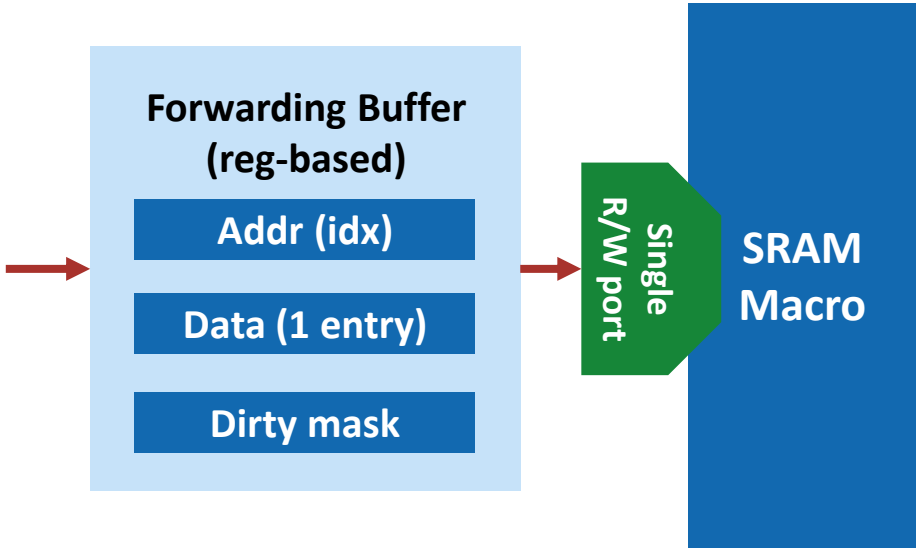
Hardware Development (Cluster)

- **Multi-Tile Backend (7nm)**
  - Old cache design (baseline)
  - Now running CTS in 7nm
  - Size: 1850x1750
    Single Tile Folded: 800x700
    Single Tile Origin: ~923x816



Hardware Development (L1 Cache)

- Forwarding Buffer implementation
  - Step 1: Transparent read-only buffer
    - 1-entry reg-based read-only cache
    - Absorbs SRAM access on buffer hit, reducing some SRAM power
    - No changing in $ ctrl logic, no performance gain
    - Status: done, all tests passed



Hardware Development (L1 Cache)

Step 2: Active write-back buffer

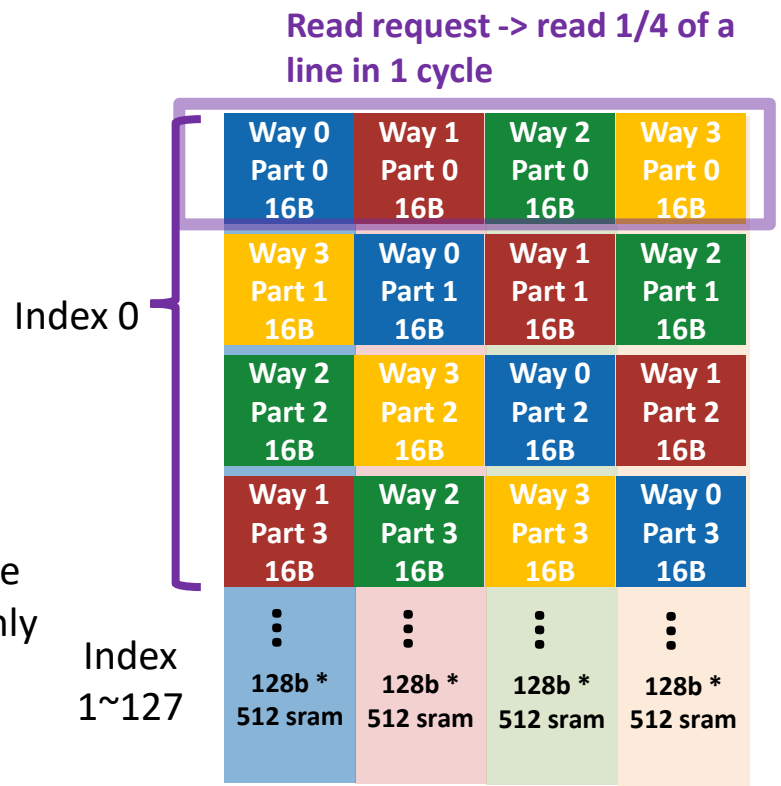
- Add handshake between the forwarding buffer and $ ctrl
  - If the buffer is writing back

    → backpressure the coming req 1 cycle
  - If the buffer hit

    → the SRAM access can finish at the same cycle
- Handle folded cache
  - The original folded cache will **access the same part of all ways**
  - Each req only **read the ¼ of the cache line** data, the following req to the same cache line but **other 3 parts will still miss** in forwarding buffer, only 5% performance improvement

  → With **hash way selection (deterministic)**, only need to access 1 way

  → As a result, we can **access all 4 parts** of the same cache line, and buffer all 4 parts into the forwarding buffer, and the following access to following 3 parts will hit



Hardware Development (L1 Cache)

Step 2: Active write-back buffer

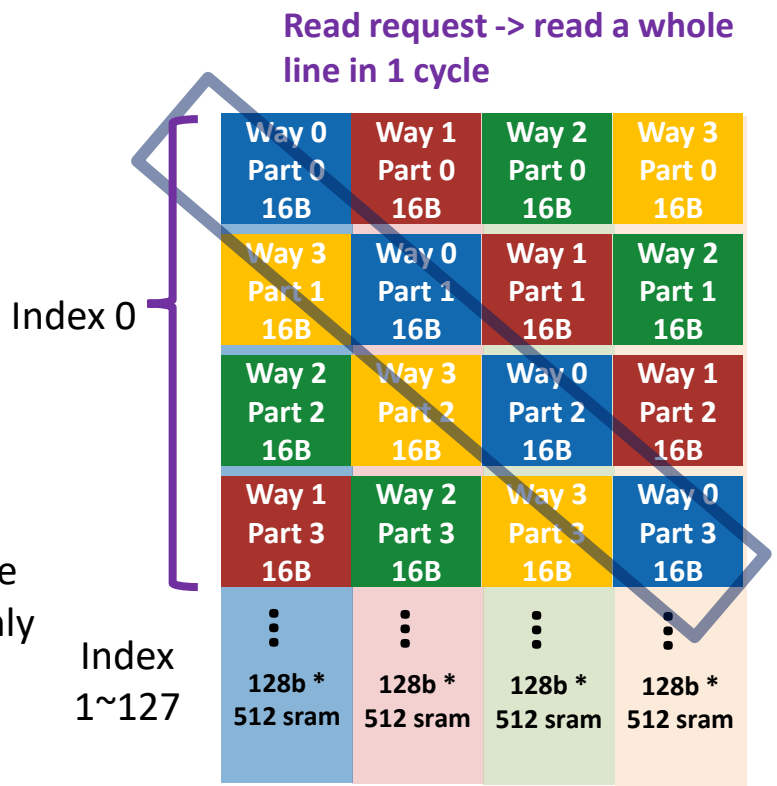
- Add handshake between the forwarding buffer and $ ctrl
  - If the buffer is writing back

    → backpressure the coming req 1 cycle
  - If the buffer hit

    → the SRAM access can finish at the same cycle
- Handle folded cache
  - The original folded cache will **access the same part of all ways**
  - Each req only **read the ¼ of the cache line** data, the following req to the same cache line but **other 3 parts will still miss** in forwarding buffer, only 5% performance improvement

  → With **hash way selection (deterministic)**, only need to access 1 way

  → As a result, we can **access all 4 parts** of the same cache line, and buffer all 4 parts into the forwarding buffer, and the following access to following 3 parts will hit



Insitu cache GVSoC model

- **Explore the existing GVSoC framework and models**
  - Reusable components from existing components:
    - cache_v3
    - hierarchical_cache
    - l1_interleaver
    - memory_v2
    - Router (interconnect model)
  - Components and functions need to be added:
    - coalescer
    - per-line pending states
    - per-set MSHR behavior
    - bank-busy tracking
    - victim selection
    - eviction handling
  - Current status
    - Implementing the model based mainly on cache_v3
    - Gradually adding the missing Insitu cache specific components

Next Step

- **InSitu-Cache Paper**
  - Both Zexin and Diyou will focus on the InSitu cache paper in May (ICCD). The progress on other side might be delayed
- **Diyou Side**
  - 7nm Flow (ongoing)
  - Merge folded cache, collect benchmark and backend results
  - RTL development on multi-group
    Based on TeraNoC structure to integrate router
- **Zexin Side**
  - Continue fixing the bandwidth issue introduced by folded cache design
  - Rebase the folded cache+forwarding buffer branch to the multi-tile main branch
  - Continue the insitu cache GVSoC model
  - Fix the new RLC kernel bugs

Timeline

		2025						2026						
		11	12	1	2	3	4	5	6	7	8	9	10	11
System	Multi-Tile Scaling [RTL] Complete													
	4/16-Group Scaling [GVSoC + RTL] Started													
L1 Access Latency	Hardware optimization (Cache & Interco) [RTL] Close to Complete													
	Cache partitioning [RTL] Complete													
	WT-L1 by master student [RTL]													
Large NoC latency	Reduce per-hop latency [Physical] Ongoing by colleague													
	Cut clk-domain (optional) [RTL + Physical]													
	NoC Topologies [RTL + GVSoC + Physical]													

Timeline

		2025						2026						
		11	12	1	2	3	4	5	6	7	8	9	10	11
Improve scalar core performance	Shared vector core [RTL + GVSoC]													
SW Kernel	Kernel optimization [SW] Delayed by one month, ongoing													
	Kernel development [SW]													
Report	Final Report and Benchmark													

L1 Access Latency

- **Problem: increased L1 access latency when scaling up**
- **Solutions**
  - **(Ongoing, RTL & Physical) Hardware optimization**
    Further optimize the interconnection and cache controller to reduce access latency on existing architecture
    => Retiming the current interconnection, remove unnecessary cuts in interco and cache
    => Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
    Currently has reduced to 6 cycles:
    Core => Xbar => AMO => Ctrl (2) => AMO => Core
  - **(Complete, RTL) Cache partitioning**
    Partition the shared L1 to keep data localized to avoid remote access latency
  - **(Ongoing, RTL & Physical) WT-L1**
    Explore and evaluate a private WT L1 for each core
    => Ongoing master thesis, now is working on the coherence between L1 caches
    => Currently plan to use hierarchical directory-based approach

L1 Access Latency

- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
  - **(Ongoing, Physical) Reduce hop latency**
    Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
    => Reduce the latency by 1/2
  - **(Planned, Physical) Cut clk-domain (optional)**
    If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
    => Effectively cut the latency by 1/3
  - **(Planned, GVSoC & Physical) NoC Topology**
    Explore different NoC topologies
    => Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2

PE-Related

- **Problem: Improve scalar core performance**
- **Solutions**
  - **(Planned, RTL & GVSoC) Shared vector core**
    Explore two Snitch sharing one Spatz configuration to improve scalar performance
    Software complexity needs to be considered, especially on the possible conflicting use of register file.
    Discussed Solution: critical section

RLC Workload

- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
  - **(Ongoing, SW) Kernel optimization**
    Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
  - **(Planned, SW) Kernel development**
    Implement the missing use cases for multi-user

Thank you!